Open Access | https://doi.org/10.25080/nwuf8465



SciPy 2025

July 7 - July 13, 2025

Proceedings of the 24th Python in Science Conference
ISSN: 2575-9752

# Reproducible Machine Learning Workflows for Scientists with Pixi

**Matthew Feickert**[1], **Ruben Arts**[2], and **John Kirkham**[3]

[1]University of Wisconsin–Madison, [2]Prefix.dev, [3]NVIDIA

## Abstract

Scientific researchers need reproducible software environments for complex applications that can run across heterogeneous computing platforms. Modern open source tools, like Pixi, provide automatic reproducibility solutions for all dependencies while providing a high level interface well suited for researchers. Combined with the recent emergence of the entire CUDA software stack — from compilers to development libraries — being supported on conda-forge, researchers are now able to easily specify their exact hardware acceleration requirements and software dependencies and get portable computational environments locked down to the digest level.

**Keywords** reproducible, machine learning, hardware acceleration, CUDA, conda-forge, Pixi

## 1. Introduction

A critical component of research software sustainability is the reproducibility of the software and computing environments software operates and "lives" in. Providing software as a "package" — a standardized distribution of all source or binary components of the software required for use along with identifying metadata — goes a long way to improved reproducibility of software libraries. However, more researchers are consumers of libraries than developers of them, but still need reproducible computing environments for research software applications that may be run across multiple computing platforms — e.g. scientific analyses, visualization tools, data transformation pipelines, and artificial intelligence (AI) and machine learning (ML) applications on hardware accelerator platforms (e.g. GPUs). While workflow engines and Linux containers offer a gold standard for scientific computing reproducibility, they require additional layers of training and software engineering knowledge. Modern open source multi-platform environment management tools, e.g. Pixi [1], provide automatic multi-platform digest-level lock file support for all dependencies — down to the compiler level — of software on public package indexes (e.g. PyPI [2] and conda-forge [3]) while still providing a high level interface well suited for researchers. Combined with the arrival of the full CUDA [4] stack on conda-forge, it is now possible to declaratively specify a full CUDA accelerated software environment. We are now at a point where well supported, robust technological solutions exist, even for applications with highly complex software environments. What is currently lacking is the education and training by the broader scientific software community to adopt these technologies and build community standards of practice around them, as well as an understanding of what are the most actionably useful features of adopting computational reproducibility tools.

## 2. Reproducibility

"Reproducible" research is a term that can mean multiple things across various fields. Some fields may view work as "reproducible" if the full process is documented, and other may

**Published** Jul 10, 2025

**Correspondence to**
Matthew Feickert
matthew.feickert@cern.ch

**Open Access**

Copyright © 2025 Feickert *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

view "reproducible" as meaning that all computations will give the same numerical outputs barring entropy variations. As there are multiple levels of reproducibility, we will restrict "reproducibility" to software environment reproducibility. We define this as be limited to the ability to define and programmatically create a software environment composed of packages that specifies all software, and its dependencies, with exact URLs and binary digests ("hashes"). Reproducible environments need to be machine agnostic in that for a specified computing platform in the environment they must be installable without modification across multiple instances.

### *2.1. Hardware accelerated environments*

Software the involves hardware acceleration on computing resources like GPUs requires additional information to be provided for full computational reproducibility. In addition to the computer platform, information about the hardware acceleration device, its supported drivers, and compatible hardware accelerated versions of the software in the environment (GPU enabled builds) are required. While this information is straightforward to collect, traditionally this has been difficult to make use of in practice given software access restrictions and the lack of declarative human interfaces for defining relationships between system-level drivers and user software. Multiple recent technological advancements (made possible by social agreements and collaborations) in the scientific open source world now provide solutions to these problems.

## 3. Conda packages

Conda packages (`.conda` files) are language agnostic file archives that contain built code distributions and metadata. This is quite powerful, as it allows for arbitrary code to be built for any target platform and then packaged with its metadata. When a conda package is downloaded and then unpacked with a conda package management tool (e.g. Pixi, conda, mamba) it is then "installed" by copying the package's file directory tree to the base of the environment's directory tree. Package contents are also simple; they can only contain files and symbolic links.

### *3.1. conda-forge*

Conda packages can be distributed on package indexes that support the concept of "channels" which redirect URLs to directory trees of conda packages. Channel names serve as the base path for hosting packages. The most broadly used community channel for conda-packages is the `conda-forge` channel, which hosts the conda packages generated from builds on the global conda-forge community cyberinfrastructure. The conda-forge community operates in a GitHub organization that hosts "feedstock" Git repositions that contain conda package build recipes as well as automation infrastructure and continuous integration (CI) and continuous delivery (CD) workflows. This allows for conda-forge community members to submit and maintain recipes — instructions for conda package build systems — to build and distribute conda packages for multiple variants of computing platforms — combinations of operating systems and hardware architectures — for Linux, macOS, and Windows.

## 4. CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA for general computing on graphical processing units (GPUs) [4], [5]. The CUDA ecosystem provides Software Development Kits (SDKs) with APIs to CUDA that allow for software developers to write hardware accelerated programs with CUDA in various languages for NVIDIA GPUs. CUDA has official language support for C++, Fortran, and Python, with community support for Julia and other languages. While there

are other types of hardware acceleration development platforms, as of 2025 CUDA is the most abundant platform for scientific computing that uses GPUs and effectively the default choice for major machine learning libraries and applications.

CUDA is closed source and proprietary to NVIDIA, which means that NVIDIA has historically limited the download access of the CUDA toolkits and drivers to registered NVIDIA developers (while keeping the software free (monetarily) to use). CUDA then required a multi-step installation process [6] with manual steps and decisions based on the target platform and particular CUDA version. This meant that when CUDA enabled environments were setup on a particular machine they were powerful and optimized, but brittle to change and could easily be broken if system wide updates (like for security fixes) occurred. CUDA software environments were bespoke and not many scientists understood how to construct and curate them.

## 5. CUDA PACKAGES ON CONDA-FORGE

### *5.1. Initial implementation*

After discussion in late 2018 [7] to better support the scientific developer community, the CUDA packaging community agreed to use the Anaconda `defaults` channel's [8] `cudatoolkit` package. Initially the `cudatoolkit` package was designed around Numba's CUDA needs [9], though it evolved to a bundle of redistributable CUDA libraries. In 2019, NVIDIA began packaging the `cudatoolkit` package in the `nvidia` conda channel. With help from the broader community, the `cudatoolkit` package was added to `conda-forge` in 2020 [10]. For the first time, this provided users the *ability to specify different versions of CUDA libraries* and download them in newly created conda environments.

Supporting initial conda-forge CUDA builds required additional components:

- A conda-forge Docker image using the NVIDIA CUDA Docker images, which provided the NVIDIA build tools for compiling packages.
- A shim package to leverage the NVIDIA build tools within a conda package build.
- A CUDA build matrix in conda-forge's global pinnings, which tied these two pieces together.

These ideas were tied together in the first package build on September 20, 2019 [11], and the initial implementation of this work was completed later in 2019. In 2020, support was expanded to Windows CUDA builds. Lots of iteration on this work happened after, all using the same basic foundation.

### *5.2. Revised implementation*

After some time using these packages and build process, a few observations became clear. First, some packages used only a subset of the libraries, like the driver, the CUDA runtime library, or particular library components like cuBLAS. However, the `cudatoolkit` package shipped considerably more than that, so having finer specifications of dependencies would provide a better package maintainer and end-user experience. Second, some packages needed components that were not part of the `cudatoolkit` bundle like other libraries or parts of the build toolchain. Having some way to depend on these components would improve usability. Third, the infrastructure management overhead of custom Docker images and their integration into the conda-forge build matrix was cumbersome for conda-forge maintainers. Being able to install and use the build tools directly would simplify maintenance and benefit end-users wishing to use these build tools.

To address these issues, NVIDIA began working on a revised set of packages. These more closely matched packages in other distribution channels (like Linux distribution package

managers) and were adapted to the conda user experience. For example, Linux distributions often install packages at the system level, which differs from the first-class userspace environment experience that conda package environments provides. As a result, some distinctions that a Linux distribution provides are unneeded in conda. There are additional differences around behavior in pinning versions of dependencies or how compilers are packaged and expected to work in their installed environments. Initial production of the packages were made on the `nvidia` channel, however, all of this work was being done internally in NVIDIA and published to a separate channel. This made the packages less visible and required additional knowledge to use.

In 2023, NVIDIA began adding the releases of CUDA conda packages from the `nvidia` channel to conda-forge, making it easier to discover and allowing for community support. Given the new package structure, NVIDIA added the packages for CUDA `12.0` to indicate the breaking change. Also with significant advancements in system driver specification support, CUDA `12` became the first version of CUDA to be released as conda packages through conda-forge and included all CUDA libraries from the CUDA compiler `nvcc` to the CUDA development libraries. CUDA metapackages were also released, which allow users to easily describe the version of CUDA they require (e.g. `cuda-version=12.5`) and the CUDA conda packages they want (e.g. `cuda`). This significantly improved the ability for researchers to easily create CUDA accelerated computing environments.

## 6. Pixi

### *6.1. Conceptual overview*

Pixi is a cross-platform package and environment manager that can handle complex development workflows [1], [12]. Importantly, Pixi automatically and non-optionally will produce or update a lock file — a structured file that contains a full list of all environments defined with a complete list of all packages, as well as definition of each packages with digest information on the binary — for the software environments defined by the user whenever any actions mutate the environment. Pixi is written in Rust, and leverages the language's speed and technologies to solve environments fast.

Pixi addresses the concept of computational reproducibility by focusing on a set of main features

1. **Virtual environment management**: Pixi can create environments that contain conda packages and Python packages and use or switch between environments easily.
2. **Package management**: Pixi enables the user to install, update, and remove packages from these environments through the `pixi` command line.
3. **Task management**: Pixi has a task runner system built-in, which allows for tasks with custom logic and dependencies on other tasks to be created.

These features become powerful when combined with robust behaviors

1. **Automatic lock files**: Any changes to a Pixi workspace that can mutate the environments defined in it will automatically and non-optionally result in the Pixi lock file for the workspace being updated. This ensures that any state of a Pixi project is trivially computationally reproducible.
2. **Solving environments for other platforms**: Pixi allows the user to solve environment for platforms other than the current user machine's. This allows for users to solve and share environment to any collaborator with confidence that all environments will work with no additional setup.

3. **Pairity of conda and Python packages**: Pixi allows for conda packages and Python packages to be used together seamlessly, and is unique in its ability to handle overlap in dependencies between them. Pixi will first solve all conda package requirements for the target environment, lock the environment, and then solve all the dependencies of the Python packages for the environment, determine if there are any overlaps with the existing conda environment, and the only install the missing Python dependencies. This ensures allows for fully reproducible solves and for the two package ecosystems to compliment each other rather than potentially cause conflicts.
4. **Efficient caching**: Pixi uses an efficient global cache shared between all Pixi projects and globally installed tools on a machine. The first time Pixi installs a package it will download the files to the global cache and link the files into the environment. When Pixi has to reinstall the same package in a different environment, the package will be linked from the same cache, making sure internet bandwidth for downloads and disk space is used as efficiently as possible.

Pixi users declaratively specify their project dependencies which are recorded in a Pixi manifest `pixi.toml` file (which for Python projects can optionally be embedded in a `pyproject.toml [pixi]` table) and automatically resolved in the `pixi.lock` lock file. This declarative nature allows for users to efficiently specify their project requirements while being guaranteed a static and reproducible environment from the lock file.

### *6.2. CUDA hardware accelerated environment creation*

Combining the features of modern CUDA `12` conda packages with Pixi's environment management, it is now possible to efficiently manage multiple software environments that can include both hardware accelerated and CPU environments. An example Pixi workspace is presented in Program 1

```toml
[workspace]
channels = ["conda-forge"]
name = "ml-example"
platforms = ["linux-64", "osx-arm64", "win-64"]
version = "0.1.0"

[tasks]

[dependencies]
python = ">=3.13.7,<3.14"

[feature.cpu.dependencies]
pytorch-cpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.cpu.tasks.train-cpu]
description = "Train a PyTorch CNN on MNIST on CPU"
cmd = "python ./src/torch_MNIST.py --epochs 2 --save-model --data-dir data"

[feature.gpu.system-requirements]
cuda = "12"

[feature.gpu.target.linux-64.dependencies]
pytorch-gpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.gpu.target.win-64.dependencies]
pytorch-gpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.gpu.tasks.train-gpu]
description = "Train a PyTorch CNN on MNIST on GPU"
cmd = "python ./src/torch_MNIST.py --epochs 14 --save-model --data-dir data"

[feature.inference.dependencies]
matplotlib = ">=3.10.3,<4"

[feature.lab.dependencies]
notebook = ">=7.4.5,<8"
jupyterlab = ">=4.4.7,<5"

[feature.lab.tasks.start]
description = "Launch JupyterLab"
cmd = "jupyter lab"

[environments]
cpu = ["cpu"]
gpu = ["gpu"]
inference = ["gpu", "inference"]
lab = ["gpu", "inference", "lab"]
```

**Program 1**. *Example of a multi-platform and multi-environment Pixi manifest with all required information and constraints to resolve and install CUDA accelerated conda packages.*

where the definition of multiple platforms allows for solving the declared environments for all platforms while on other platforms

```toml
[workspace]
channels = ["conda-forge"]
name = "ml-example"
platforms = ["linux-64", "osx-arm64", "win-64"]
version = "0.1.0"
```

the `cpu` feature defines `dependencies` and `tasks` that are accessible from the `cpu` environment

```
...

[feature.cpu.dependencies]
pytorch-cpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.cpu.tasks.train-cpu]
description = "Train a PyTorch CNN on MNIST on CPU"
cmd = "python ./src/torch_MNIST.py --epochs 2 --save-model --data-dir data"

...

[environments]
cpu = ["cpu"]
```

The `gpu` feature does the same for the `gpu` environment, but it also importantly defines a `system-requirements` table that define the system specifications needed to install and run a Pixi workspace's environments.

```
...

[feature.gpu.system-requirements]
cuda = "12"

[feature.gpu.target.linux-64.dependencies]
pytorch-gpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.gpu.target.win-64.dependencies]
pytorch-gpu = ">=2.7.1,<3"
torchvision = ">=0.22.0,<0.23"

[feature.gpu.tasks.train-gpu]
description = "Train a PyTorch CNN on MNIST on GPU"
cmd = "python ./src/torch_MNIST.py --epochs 14 --save-model --data-dir data"

...

[environments]
...
gpu = ["gpu"]
```

`system-requirements` build upon the concept of conda "virtual packages", allowing for the dependency resolver to enforce constraints declared by defining compatibility of the system with virtual packages, like `__cuda`. In the particular case of CUDA, the `system-requirements` table specifies the CUDA version the workspace expects the host system to support, as detected through the host system's NVIDIA driver API. While the `system-requirements` field values do not correspond to lower or upper bounds, specifying that the workspace is expected to work on systems that support CUDA 12

```
...

[feature.gpu.system-requirements]
cuda = "12"

...
```

ensures that packages depending on `__cuda >= 12` are resolved correctly. This effectively means that declaring the system requirement will cause the Pixi dependency resolver to find CUDA enabled packages that are compatible with CUDA 12, disallowing for incompatible package builds to be resolved. Once these package dependencies have been resolved and locked, this ensures that any system capable of meeting the system requirement will get working CUDA accelerated conda packages installed.

Not all machines will have an NVIDIA GPU on them to allow for the system requirements to be resolved correctly. To allow for non-CUDA-supported-machines to still resolve Pixi workspace requirements, shell environment overrides exist through the `CONDA_OVERRIDE_CUDA` environmental variable. Setting `CONDA_OVERRIDE_CUDA=12` on a machine that doesn't meet the CUDA version requirements, will override the supported virtual packages and set a value of `__cuda=12` for the system. This can be clearly understood from setting the override and then querying the workspace summary with `pixi info`, as seen in Program 2. This is a powerful functionality as it allows for environment specification, resolution, and locking for target platforms that users might not have access to, but can be assured are valid.

```
% pixi info
System
------------
       Pixi version: 0.54.1
           Platform: osx-arm64
   Virtual packages: __unix=0=0
                   : __osx=15.3.2=0
                   : __archspec=1=m2
...

% CONDA_OVERRIDE_CUDA=12 pixi info
System
------------
       Pixi version: 0.54.1
           Platform: osx-arm64
   Virtual packages: __unix=0=0
                   : __osx=15.3.2=0
                   : __cuda=12=0
                   : __archspec=1=m2
...
```

**Program 2.** *Demonstration of using the `CONDA_OVERRIDE_CUDA` environmental variable on a system with no CUDA support (an Apple silicon machine) to allow dependency resolution as if it supported CUDA 12.*

Pixi also allows for feature composition to efficiently create new environments. Program 1's `gpu` and `inference` features are combined and resolved collectively to provide a new CUDA accelerated `inference` environment that does not affect the `gpu` environment. The same applies for the `lab` feature and environment, which additionally provides JupyterLab for interactive programming with notebooks.

```
...

[feature.inference.dependencies]
matplotlib = ">=3.10.3,<4"

[feature.lab.dependencies]
notebook = ">=7.4.5,<8"
jupyterlab = ">=4.4.7,<5"

...

[environments]
...
gpu = ["gpu"]
inference = ["gpu", "inference"]
lab = ["gpu", "inference", "lab"]
```

Composing multiple environments from Pixi features allows for separating conceptual steps of scientific analysis into bespoke software environments that contain only the necessary dependencies. This allows for each step's environment to be better defined, potentially with radically different or conflicting dependencies from other steps, and for clean separation between interactive and non-interactive ("batch") computing models.

### *6.3. Locked environments*

Once the workspace has been defined, any Pixi operation on the workspace will result in all environments in the workspace having their dependencies resolved and then fully specified ("locked") at the digest ("hash") level in a single `pixi.lock` Pixi lock file, as seen in Program 3. The lock file is a YAML file that contains two definition groups: `environments` and `packages`. The `environments` group lists every environment in the workspace for every platform with a complete listing of all packages in the environment. The `packages` group lists a full definition of every package that appears in the `environments` lists, including the package's URL and digests (e.g. sha256, md5). These groups provide a full description of every package described in the Pixi workspace and its dependencies and constraints on other packages. Versioning the lock file along with the manifest file in a version control system allows for workspaces to be fully reproducible to the byte level indefinitely into the future, conditioned on the continued existence of the package indexes the workspace pulls from (e.g. conda-forge, PyPI, the nvidia conda channel). In the event that long term preservation and reproducibility are of importance, there are community projects [13] that allow for downloading all dependencies of a Pixi environment and generating a tar archive containing all of the packages, which can later be unpacked and installed.

```
version: 6
environments:
  cpu:
    channels:
    - url: https://conda.anaconda.org/conda-forge/
    packages:
      linux-64:

...

      - conda: https://conda.anaconda.org/conda-forge/linux-64/python-3.13.7-h2b335a9_100_cp313.conda
      - conda: https://conda.anaconda.org/conda-forge/noarch/python_abi-3.13-8_cp313.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/pytorch-2.7.1-cpu_mkl_py313_h58dab0e_
103.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/pytorch-cpu-2.7.1-cpu_mkl_hc60beec_
103.conda

...

  gpu:
    channels:
    - url: https://conda.anaconda.org/conda-forge/
    packages:
      linux-64:

...

      - conda: https://conda.anaconda.org/conda-forge/linux-64/cuda-nvcc-tools-12.9.86-he02047a_2.
conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/cuda-nvdisasm-12.9.88-hbd13f7d_0.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/cuda-nvrtc-12.9.86-h5888daf_0.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/cuda-nvtx-12.9.79-h5888daf_0.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/cuda-nvvm-tools-12.9.86-h4bc722e_2.
conda
      - conda: https://conda.anaconda.org/conda-forge/noarch/cuda-version-12.9-h4f385c5_3.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/cudnn-9.10.1.4-hbcb9cd8_1.conda

...

      - conda: https://conda.anaconda.org/conda-forge/linux-64/python-3.13.7-h2b335a9_100_cp313.conda
      - conda: https://conda.anaconda.org/conda-forge/noarch/python_abi-3.13-8_cp313.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/pytorch-2.7.1-cuda129_mkl_py313_h1e53
aa0_304.conda
      - conda: https://conda.anaconda.org/conda-forge/linux-64/pytorch-gpu-2.7.1-cuda129_mkl_h43a4b0
b_304.conda

...

packages:

...

- conda: https://conda.anaconda.org/conda-forge/linux-64/pytorch-gpu-2.7.1-cuda129_mkl_h43a4b0b_304.
conda
  sha256: af54e6535619f4e484d278d015df6ea67622e2194f78da2c0541958fc3d83d18
  md5: e374ee50f7d5171d82320bced8165e85
  depends:
  - pytorch 2.7.1 cuda*_mkl*304
  license: BSD-3-Clause
  license_family: BSD
  size: 48008
  timestamp: 1753886159800

...
```

**Program 3.** *Example structure of a `pixi.lock` Pixi lock file showing the definition of the environments as well as a full description of each package used in each environment.*

## 7. Deploying environments to remote compute

Often researchers are running scientific and machine learning workflows on remote computational resources that use batch computing systems (e.g. HTCondor, SLURM). For systems

with shared filesystems (e.g. SLURM) it is possible to use Pixi workspaces in workflows in a similar manner to local machine (e.g. laptop or workstation). Other systems (e.g. HTCondor) do not have a shared filesystem (e.g. HTCondor), requiring that each worker node receive its own copy of the software environment. While locked Pixi environments significantly help with this, it is often advantageous to distribute the environment in the form of a Linux container image to the compute resources. These systems are able to mount Linux container images to worker nodes in ways that reduce the disk and memory cost to the user's session, compared to installing Pixi and then downloading all dependencies of the software environment from the package indexes used. This also reduces the bandwidth use as the Linux container image can be cached at the compute resource host and efficiently replicated to the worker nodes, paying the bandwidth cost of download once. While Linux container technology historically has presented additional engineering and design overhead to researchers, Linux container construction of Pixi environments is simple and can be reduced to templated format. An example in the form of a templated Dockerfile is seen in Program 4.[1] The template requires user input to define the target CUDA version (`CUDA_VERSION`) and the name of the Pixi environment to install (`ENVIRONMENT`). As the Pixi environment is already fully defined and locked it can be directly installed as normal in the `build` stage of the container image build, along with an entrypoint shell script that will activate the environment, and then copied from the `build` stage into the `final` stage to reduce the total image size by removing the cache and reducing the total number of layers in the final image.

```
/ml-example/Dockerfile
ARG CUDA_VERSION="12"
ARG ENVIRONMENT="gpu"

FROM ghcr.io/prefix-dev/pixi:noble AS build

# Redeclaring ARGS in a stage without a value inherits the global default
ARG CUDA_VERSION
ARG ENVIRONMENT

WORKDIR /app
COPY . .
ENV CONDA_OVERRIDE_CUDA=$CUDA_VERSION
RUN pixi install --locked --environment $ENVIRONMENT
RUN echo "#!/bin/bash" > /app/entrypoint.sh && \
    pixi shell-hook --environment $ENVIRONMENT -s bash >> /app/entrypoint.sh && \
    echo 'exec "$@"' >> /app/entrypoint.sh

FROM ghcr.io/prefix-dev/pixi:noble AS final

ARG ENVIRONMENT

WORKDIR /app
COPY --from=build /app/.pixi/envs/$ENVIRONMENT /app/.pixi/envs/$ENVIRONMENT
COPY --from=build /app/pixi.toml /app/pixi.toml
COPY --from=build /app/pixi.lock /app/pixi.lock
# The ignore files are needed for 'pixi run' to work in the container
COPY --from=build /app/.pixi/.gitignore /app/.pixi/.gitignore
COPY --from=build /app/.pixi/.condapackageignore /app/.pixi/.condapackageignore
COPY --from=build --chmod=0755 /app/entrypoint.sh /app/entrypoint.sh

ENTRYPOINT [ "/app/entrypoint.sh" ]
```

**Program 4**. *The template structure of a Dockerfile for a locked Pixi environment with CUDA dependencies. The only values that need user input are the CUDA version and the name of the target environment.*

[1] As many compute facilities do not allow for use of Docker directly given security concerns, Apptainer container image formats are more common. Apptainer definition files are similarly easy to write as compared to Dockerfiles and Docker container images can be converted into a format that Apptainer can use. As Docker is a more common format in the broader computing world, including commercial settings, it has been used for this example. These workflows are not limited to a single container image format.

The Dockerfile can then be built into a Linux container image binary file which can be distributed to a container image registry. Batch computing system workflow definition files can use these container images to provide the software environment for the computing jobs, which pull the images from the container image registry when requested by the job.

## 8. Summary

As hardware accelerated code becomes more common across scientific computing, especially CUDA accelerated software for machine learning, the need for simple but powerful solutions for software environment management has grown too. The simple and flexible structure of conda packages allows for complex projects to be packaged as directory trees of built binaries on a platform specific level. This has allowed for the complexity of the CUDA software stack to be efficiently built as conda packages using the conda-forge cyberinfrastructure and then distributed on the conda-forge conda channel for public use. Distribution of CUDA conda packages on conda-forge additionally allows for other conda-forge projects to use CUDA conda packages in their builds, resulting in a wide selection of CUDA enabled projects, including many machine learning packages. Through use of Pixi's declarative specification of dependencies in the project manifest and non-optional digest level lock file generation, software environments can now be declaratively and rapidly constructed, resolved, and locked using semantic operations well designed for scientific researchers. With these powerful technologies and abstractions, researchers can now construct machine learning and data science environments for multiple platforms at once and use trusted patterns to develop locally and deploy to remote computational resources.

In addition to the long term reproducibility provided by the combination of these technologies, the maintenance burden and complexity reduction should not be overlooked. With the CUDA v12 distributions on conda-forge, researches no longer need to have experience in CUDA internals and distribution installation to accelerate their software projects. They need only know the supported versions of CUDA by the NVIDIA drivers on their target machines. Researchers also no longer need to use multiple tools to build bespoke workflows for constructing and maintaining lock files for multiple environments and platforms, while keeping environment definition files and lock files synced. Pixi provides a single tool and unified interface to achieve the same results faster while using high level abstractions — removing most of the work of software environment reproducibility from the user workflow.[2] Having the full specification of the software environment including the CUDA dependencies also removes runtime failures due to missing, unspecified, or incompatible system-level requirements on remote compute resources. Most importantly, reducing cognitive overhead and the latency to reach a usable software environment reduces the time to insight for researchers, transferring the problems of scientific computing back into their domains of expertise.

### Acknowledgements

Matthew Feickert is supported by the U.S. National Science Foundation (NSF) under Cooperative Agreement PHY-2323298 (IRIS-HEP) and by the US Research Software Sustainability Institute (URSSI) via grant G-2022-19347 from the Sloan Foundation. Ruben Arts is supported by prefix.dev GmbH. John Kirkham is supported by NVIDIA. The described work [14] was created in association with [15].

[2] It is worth reflecting on that the auspicious interplay of these technologies is a recent advancement that could not have happened significantly earlier. Conda-forge was created as a project in 2015 (at the SciPy 2015 conference), and the first distributions of CUDA v12 were in 2022, with Pixi being created in 2023. The advancements that have occurred in less than two years after their coexistence is a reflection of the power of strong design standards and collaboration across the conda-forge community.

## References


[1] R. Arts, B. Zalmstra, W. Vollprecht, T. de Jager, N. Morcotilo, and J. Hofer, "pixi." [Online]. Available: https://github.com/prefix-dev/pixi/releases/tag/v0.54.1

[2] "The Python Package Index (PyPI)." [Online]. Available: https://pypi.org/

[3] conda-forge community, "The conda-forge Project: Community-based Software Distribution Built on the conda Package Format and Ecosystem." [Online]. Available: https://doi.org/10.5281/zenodo.4774216

[4] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," in *ACM SIGGRAPH 2008 Classes*, in SIGGRAPH '08. Los Angeles, California, 2008. doi: 10.1145/1401132.1401152.

[5] M. Fatica, "CUDA toolkit and libraries." [Online]. Available: https://doi.org/10.1109/HOTCHIPS.2008.7476520

[6] "CUDA Installation Guide for Linux." [Online]. Available: https://docs.nvidia.com/cuda/cuda-installation-guide-linux/

[7] "conda-forge.github.io Issue 687: How to specify CUDA version in a conda package?." [Online]. Available: https://github.com/conda-forge/conda-forge.github.io/issues/687

[8] Anaconda, Inc., "Anaconda Documentation: Default channels." (https://www.anaconda.com/docs/tools/working-with-conda/reference/default-channels, 2025.

[9] Continuum Analytics, Inc., "numba/conda-recipe-cudatoolkit." [Online]. Available: https://github.com/numba/conda-recipe-cudatoolkit

[10] "conda-forge/staged-recipes Pull Request 12882: Cudatoolkit." [Online]. Available: https://github.com/conda-forge/staged-recipes/pull/12882

[11] "conda-forge/ucx-split-feedstock Pull Request 14: Add GPU builds." [Online]. Available: https://github.com/conda-forge/ucx-split-feedstock/pull/14

[12] R. Arts, B. Zalmstra, W. Vollprecht, T. de Jager, N. Morcotilo, and J. Hofer, "Pixi Documentation." [Online]. Available: https://pixi.sh/v0.54.1/

[13] P. Zwerschke, D. Elsner, and B. Stoyan, "pixi-pack." [Online]. Available: https://github.com/Quantco/pixi-pack/releases/tag/v0.7.2

[14] M. Feickert, R. Arts, and J. Kirkham, "SciPy 2025 Tutorial: Reproducible Machine Learning Workflows for Scientists with Pixi." [Online]. Available: https://github.com/matthewfeickert-talks/reproducible-ml-for-scientists-with-pixi-scipy-2025/releases/tag/scipy-2025

[15] M. Feickert, "Reproducible Machine Learning Workflows for Scientists." [Online]. Available: https://github.com/carpentries-incubator/reproducible-ml-workflows